音速并行：基于 Artix-7 的纯 RTL 音频一体化系统
技术设计报告

音速并行（Sonic-Speed-Parallelism）

——纯 RTL 音频采集 · 均衡 · 频谱分析 · 双路可视化一体化系统

摘　要

本作品在 Xilinx Artix-7 XC7A100T 上以**纯 RTL**（不使用 HLS、软核或任何现成 IP，连 TMDS 编码与 5:1 串行化也由原语手写）实现了一套音频一体化系统，覆盖“采集 → 均衡 → 频谱分析 → 可视化 → 交互”的完整闭环。

系统以 WM8960 编解码器为音频前端（48 kHz / 24 bit），采样数据同时进入两条并行流水线：一是**5 段定点 IIR 参量均衡器**，逐段做 Q 格式与位宽预算，各段增益可在线独立调节并实时回放；二是**1024 点基 2 FFT 频谱分析引擎**，仅用一个蝶形单元配合双口 BRAM 时分复用，硬件开销不到全并行方案的 1/512，整帧处理仅占帧周期的 1.3%。频谱结果经对数频率分箱、峰值保持与线性衰减后驱动**双路独立显示**：LCD 480×272 作交互界面（8 按钮面板 + 8 行状态栏），HDMI 640×360 作展示大屏（80 根独立柱 + 波形），并从 SD 卡读取图片作为动态背景（LCD 1:1 满屏、HDMI 裁中心 320×180 后 2 倍整数放大）。

系统横跨四个时钟域，以格雷码异步 FIFO 与电平翻转握手完成跨域传递。所有参数可经板载按键或 UART ASCII 命令在一个时钟周期内生效。

整机占用 11 716 LUT（18.48%）、100.5 BRAM、26 DSP48，时序全部收敛（`clk_sys` +1.779 ns / `clk_hdmi` +4.028 ns），**综合与实现 0 错误 0 警告**，39 个测试平台全部与 Python 定点黄金模型逐位对拍通过，并已通过 QSPI Flash 固化实现掉电自启动。

关键词：FPGA；纯 RTL；定点 DSP；频谱可视化；HDMI；跨时钟域

目录

摘要 …… 1
1 第一部分　作品概述 …… 4
1.1 选题背景与实际问题 …… 4
1.2 功能与特性 …… 5
1.3 主要技术特点及创新点 …… 7
1.4 主要性能指标 …… 8
1.5 设计流程 …… 8
2 第二部分　系统组成及功能说明 …… 11
2.1 整体介绍 …… 11
2.2 硬件系统介绍 …… 12
2.2.1 硬件整体介绍 …… 12
2.2.2 机械设计介绍 …… 13
2.2.3 电路各模块介绍 …… 13
2.3 软件系统介绍 …… 15
2.3.1 软件整体介绍 …… 15
2.3.2 软硬件划分依据 …… 17
2.3.3 软件各模块介绍 …… 17
3 第三部分　完成情况及性能参数 …… 20
3.1 整体介绍 …… 20
3.2 工程成果 …… 21
3.2.1 机械成果 …… 21
3.2.2 电路成果 …… 22
3.2.3 软件成果 …… 22
3.3 特性成果（量化指标） …… 23
4 第四部分　总结 …… 27
4.1 大模型协作记录 …… 27
4.1.1 所用 AI 工具与使用环节 …… 27
4.1.2 主要提示方式 …… 27
4.1.3 自我纠错轨迹（五个典型案例） …… 29
4.1.4 智能体工作流设计 …… 29
4.1.5 采纳、人工修改与核验 …… 30

4.2 技能包的提炼过程 …… 31
4.2.1 三级沉淀体系 …… 31
4.2.2 四个技能包的来源、验证与边界 …… 31
4.2.3 从“观察”到“技能”的抽象链路 …… 32
4.2.4 边界如何确定 …… 32
4.3 可扩展之处 …… 33
4.4 心得体会 …… 33
5 第五部分　参考文献 …… 35
A 附录：复现说明 …… 36
1.1 环境要求 …… 36
1.2 从零开始的完整步骤 …… 36
1.2.1 步骤 1：获取代码 …… 36
1.2.2 步骤 2：工具链自检 …… 36
1.2.3 步骤 3：全量仿真（约 1 分 30 秒） …… 37
1.2.4 步骤 4：静态检查 …… 37
1.2.5 步骤 5：综合与实现（约 9~10 分钟） …… 37
1.2.6 步骤 6：准备 SD 卡（需要 sudo） …… 37
1.2.7 步骤 7：上板 …… 37
1.3 常见问题 …… 38

1 第一部分　作品概述

1.1 选题背景与实际问题

音频可视化设备在消费市场需求广泛，例如车载音响频谱显示、直播/演出调音台可视化、Hi-Fi 播放器界面等等。此外，在工业检测领域：如机械振动频谱监测、电机异响诊断、齿轮箱故障早期预警等案例中也经常依赖于频谱分析方法。虽然目前音频可视化技术已经非常成熟，但现有产品存在一个普遍的结构性矛盾，如表 1 所示：

表 1　现有方案的结构性矛盾

方案	成本	主要问题
PC + 声卡 + 上位机	低	延迟高（10~100 ms），功耗大，不适合嵌入式部署
MCU + CMSIS-DSP	低	算力不足，1024 点 FFT + 5 段 EQ + 双屏驱动无法同时满足
Zynq + PYNQ	中	依赖 HLS/Python 框架，底层时序不可控，启动秒级

核心矛盾在于“实时性”与“可调性”难以兼得。专用 ASIC 实时但不灵活，软件方案灵活但延迟大。本作品主要要解决的问题以及和目前主流方案的对比如下：

本作品要解决的三个实际问题

1. **确定性延迟**：从 ADC 采样到屏幕像素，延迟以时钟周期计，不受操作系统调度、缓存命中率影响；
2. **在线可调**：所有参数（EQ 各段增益、配色、刷新率、视图）可经按键或 UART 进行配置，可在一个时钟周期内生效；
3. **零 IP 依赖**：从 I2C 主机、I2S 从机、FFT 蝶形、存储器控制器、TMDS 编码器到 5:1 串行化，全部手写，可逐行审计。

表 2　与已有工作的对比

方向	代表做法	本作品的差异
FPGA 音频 EQ	Xilinx FIR/IIR Compiler IP	手写二阶节级联，逐段做 Q 格式与位宽预算
FPGA FFT	Xilinx FFT IP（AXI4-Stream）	手写基 2 DIT + 单蝶形时分复用，资源为全并行方案的 1/512
频谱可视化	固定柱数、PC 侧渲染	扫描式方案，无 framebuffer，关键配置全参数化
FPGA HDMI	现成 IP 或 VHDL 开源核	手写 TMDS 编码器 + OSERDESE2 原语
开发方法	传统的 Vivado 构建方法	大模型协作 + skill + tcl 脚本

1.2 功能与特性

本作品是一套从模拟前端到双路显示输出的完整闭环系统，而非单个算法模块的演示。系统的功能可分为四个层次，如表 3 所示。**音频前端**负责把模拟信号变成可处理的数字样本，**分析引擎**把时域样本变成频域柱高与波形，**可视化层**把抽象数据变成可看的图像，**交互与存储**则负责参数与素材的进出。

表 3　系统功能清单（按四个层次组织）

层次	功能	状态	说明
音频前端	话筒拾音	✓	板载 MEMS 麦克风（LINPUT1），48 kHz / 24 bit
	5 段参量均衡	✓	二阶节级联，各段 −10 ∼ +10 dB 独立可调，25 条系数在线重载
	双声道输出	✓	WM8960 DAC 双声道，数字音量 Q1.7
	SPK 静音	✓	送全 0 采样，`data_en` 不受影响，I2S 时序稳定不爆音
	模拟旁路关闭	✓	修 R45/R46 的 0 dB 旁路默认开启问题，否则 DSP 全程无效
分析引擎	1024 点 FFT	✓	基 2 DIT，单蝶形时分复用，2048 频点逐位可核
	幅度估计	✓	$\max + \frac{7}{16}\min$ 近似，误差 < 0.6 dB
	对数频率分箱	✓	按 Bark 尺度分箱，60 根（LCD）/ 80 根（HDMI）两套表
	峰值保持 + 衰减	✓	每帧线性衰减，柱高不会抖
可视化层	三种视图	✓	柱状频谱 / 极坐标频谱 / 动态波形，可在同一屏叠加
	扫描式渲染	✓	$(x, y, \text{mode}) \to$ RGB 纯组合函数，LCD 侧零 framebuffer
	彩虹配色	✓	128 级彩虹表，色相漂移速度 4 档可调
	波形触发	✓	迟滞触发 + 自适应门限（带上下限钳位）+ 刷新率可降
	双路输出	✓	LCD 480×272@12.5 MHz + HDMI 640×360@25 MHz，设置自动同步
交互与存储	8 按钮面板	✓	VIEW / SPK / HUE / GAIN / BG / DEMO / IN / SD
	8 行状态栏	✓	每行 10 字符，实时显示全部配置状态
	UART 命令通道	✓	115200 8N1，ASCII 命令改任意配置并回执
	SD 图片 + Flash	✓	3 张图循环（LCD 满屏 1:1 / HDMI 裁剪后 2 倍放大）；QSPI 掉电自启动

在功能清单之上，本作品有三个值得单独展开的设计选择，也体现了在实际工程中容易遇到的一些问题以及解决的方法：

（一）扫描式双路渲染：一份核心，两块屏

常规做法是先渲染到帧缓冲，再由扫描逻辑读出，但 640×360×16 bit 的帧缓冲需要 103 块 BRAM，加上其它模块更是直接达到了 98% 左右，而本器件只有 135 块，实测会导致布线拥塞、时序更是崩到 -1.09 ns、导致 `place_design` 构建失败。

本作品把渲染核心抽象为纯组合函数 $(x, y, \text{mode}) \to \text{RGB}$：

```
// 同一个模块，靠参数决定分辨率和是否画 UI
disp_top #(.HDISP(480), .VDISP(272), .WITH_UI(1)) u_disp_lcd  (...);  // 12.5 MHz
disp_top #(.HDISP(640), .VDISP(360), .WITH_UI(0)) u_disp_hdmi (...);  // 25 MHz
```

这样带来的好处主要有三重：1. LCD 侧零 framebuffer，只有 SD 图片需要存储；2. 换分辨率只改参数，不改逻辑；3. 两路的配色、柱高、波形天然一致，因为它们看的是同一份输入数据。

（二）数据柱数与显示柱数解耦

LCD 的像素宽为 480，使用 60 根柱则每柱 8 像素，可以整除；而 HDMI 的像素宽为 640，若也用 60 根柱则每柱 10.67 像素，无法整除，所以必须用乘法器，但是乘法器会恶化时序。

本作品把数据大小和显示柱数拆成两个独立参数（`NBARS` / `NBARS_DISP`），HDMI 侧改用 80 根柱可以整除，只需移位。更关键的是原本路径由 `generate` 的恒等分支保护：

```
generate
    if (NBARS_DISP == NBARS) begin : g_ident
        assign bbar = dbar[AW-1:0];              // 综合器看不到除法
    end else begin : g_scale
        assign bbar = (dbar * NBARS) / NBARS_DISP;
    end
endgenerate
```

（三）三个细节层面的设计

- **静音改用“送 0 数据”而不是“停 I2S”**：`data_en` 照常拉高，只是把左右声道数据清零。这样 WM8960 收到的是“合法的 0 采样数据”，链路不会以为对端掉线，避免由静音或取消静音而产生的瞬间爆音。
- **跨域握手用“电平翻转”而不是“单拍脉冲”**：`start` 若是单拍脉冲，跨到更慢的时钟域时可能被整个漏掉；改成翻转一个电平再双打取边沿，无论两侧时钟比多大都不会丢。
- **按键活动标志也用电平翻转**：面板的空闲自动收起计时器需要知道“用户刚才是否有动作”，同样不能用一个拍宽的脉冲跨域。

1.3 主要技术特点及创新点

1. 单蝶形 FFT 的极低资源占用

1024 点 FFT 若全并行需 512 个蝶形 × 4 乘法器 = **2048 个 DSP48**，超出器件总量（240 个）近 8 倍。而本作品用一个蝶形单元加双口 BRAM 进行时分复用：

表 4　FFT 架构方案对比（1024 点，48 kHz 采样）

方案	蝶形单元	DSP48	BRAM	整帧耗时	占帧周期
全并行	512	2048	0	1 拍	0.04%
4 路并行	128	512	0	7 680 拍	0.3%
单蝶形（采用）	**1**	**4**	**2**	**30 720 拍**	**1.3%**

WM8960 采样数据一帧周期大约为 46.9 ms，而 FFT 计算整帧耗时 30 720×20.83 ns = 0.64 ms，仅占 1.3%，余量为 75 倍。

2. 扫描式渲染，无帧缓冲

把渲染核心抽象为纯组合函数 $(x, y, \text{mode}) \to \text{RGB}$，扫描到哪个像素就算哪个像素。LCD 侧无需任何 framebuffer，一帧 130 560 个像素全部实时算出。

3. 零成本的裁剪与放大

HDMI 侧需要从 480×272 的全图中裁出中心 320×180 的区域，再放大 2 倍铺满 640×360。在实际操作中，可以通过两次移位相加从而避免使用到乘法器：

```
// 裁剪地址：dy*320 = dy*256 + dy*64 -> 两次移位相加，零 DSP
wire [18:0] crop_addr = (dy << 8) + (dy << 6) + dx;

// 2 倍放大：显示坐标右移 1 位 -> 一个移位，零算术
wire [XW-1:0] fb_sx = (fb_lx >> 1) + FB_CX[XW-1:0];
wire [YW-1:0] fb_sy = (fb_ly >> 1) + FB_CY[YW-1:0];
```

4. 不同尺寸双帧缓冲替代“双读口”

两个显示器时钟不同（12.5 MHz / 25 MHz）。实测表明 Vivado 的 BRAM 推断不支持“1 写 2 读跨时钟”，会静默退化成分布式 RAM（LUTRAM），触发 `DRC UTLZ-1` 错误（需 86 452 个 `RAMD64E`，仅 19 000 个可用），加 `(* ram_style = "block" *)` 进行约束也无效。

因此本作品改为两份不同尺寸的缓冲，即 LCD 侧直接读取 480×272 的全部图像数据，HDMI 读取对应的 320×180 范围内的裁剪区图像数据后进行 2 倍整数放大，这样

一来 BRAM 从 116 块降到 84 块，节省一定资源的同时也避开了由静默推断而导致的错误。

1.4 主要性能指标

表 5　主要性能指标

类别	指标	值
资源	Slice LUTs	11 495 / 63 400（18.13%）
	Block RAM	100.5 / 135（74.44%）
	DSP48	26 / 240（10.83%）
	MMCM	2 / 6（33.33%）
时序	`clk_sys`（48 MHz）	WNS **+1.779 ns**
	`clk_hdmi`（25 MHz）	WNS **+4.028 ns**
	`clk_pix`（12.5 MHz）	WNS 远大于 0
实时性	FFT 整帧耗时	0.64 ms（占帧周期 1.3%）
	端到端延迟	≈82 ms（80 ms 为 LCD 一帧扫描）
质量	综合/实现警告	**0 CRITICAL / 0 WARNING**
	静态检查	Verible 0 / Verilator 0
	仿真回归	**39 / 39 PASS**
显示	LCD / HDMI	480×272@12.5 MHz / 640×360@25 MHz
音频	采样 / 均衡	48 kHz / 24 bit / 5 段双二阶

1.5 设计流程

本作品采用的不是”写完代码再补测试”的传统流程，而是**模型先行、逐位对拍、每轮全量回归**的闭环流程。这样做是因为 RTL 调试的最大成本不在”写”、而在”怎么确认它是对的”——尤其是定点算法，肉眼根本看不出 1 LSB 的偏差。

整个流程分为**五条并行推进的线索**，如图 1 所示：

1. **规格与判据先行**：在写任何 RTL 之前，先把”怎样算对”写成可执行的判据。例如”SD 卡初始化的第一次 `Go` 必须走完 5 步、**第二次 `Go` 也必须能重新走完**”——这条判据后来救了大命（见第四部分 E 系列案例）。
2. **黄金模型先行**：每个定点模块先用 Python（`numpy` / `mpmath`）写出”数学上正确”的参考实现，再做定点量化与误差评估，确认位宽够用后，才由脚本生成 RTL 需要的

系数表 / 旋转因子表 / 字模。

3. **RTL 与 Testbench 同步编写**：写一个模块、配一个 testbench，两者一起提交。testbench 的判据直接取自第 1 步；激励采用” 三段式”（复位段 / 正常段 / 边界段），边界段必须覆盖” 第二次触发” 这类容易漏的路径。
4. **每轮全量回归**：改完任何一个文件，都跑一次 39 个 testbench 的并行回归（约 1 分 30 秒），而不是” 只跑相关的那几个”。曾经因为只跑相关 TB，漏掉了一个跨域信号的 `UNDRIVEN`，直到上板才发现。
5. **综合报告是第二道关**：仿真全绿只是” 逻辑对”，还要看资源是否超限（BRAM >80% 就要重新设计）、时序是否全为正（**必须看时钟对表，不能只看域内 WNS**）、DRC 是否为零。三者都过才允许上板。

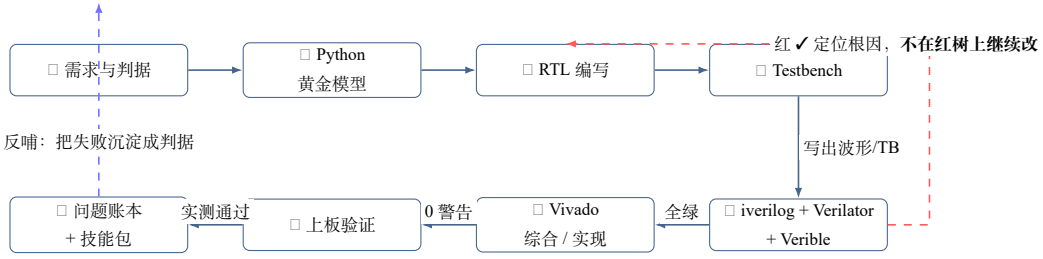


图 1　设计流程：五条线索并行推进的闭环（蓝虚线为知识反哺，红虚线为失败回退）

流程中有两个” 不对称” 值得特别说明：

不对称一：仿真红与综合红，处理方式完全不同

- **仿真红**（第 □ 步 □ 回第 □ 步）：这是**逻辑错**，可以定位后继续改。但有一个硬规则——**同一问题连猜 3 次不中就必须停下来**，用 `git checkout --` 回退到上一个绿状态重新分析。在红树上继续打补丁，只会把问题越搅越乱（这个教训在 SD 卡初始化调试时付出了一整天代价）。
- **综合红**（资源超限 / 时序为负 / DRC 非零）：这通常**不是 bug，而是架构问题**。例如 BRAM 占用到 98% 导致布线拥塞、进而 `clk_sys` 违例 −1.09 ns、`place_design` 直接失败——这时候改代码没用，必须**改架构**（换存储方案 / 降分辨率 / 拆模块）。

不对称二：账本反哺的是” 下一轮的规格”，不是” 这一轮的代码”

问题账本（825 条）和技能包（4 个）不是” 事后总结”，而是在**下一轮设计的第 □ 步就被读取**。例如在做 SD 卡子系统之前，先读了 `rtl-verification` 技能包里” 永远不要在信号自己更新的沿上读它” 这一条，于是在写 `sd_read` 的 testbench 时就**避免**了 `@(negedge sd_clk)` 这个坑——这是把过去的失败**前置**成设计约束，

而不是等它再犯一次。

2 第二部分　系统组成及功能说明

2.1 整体介绍

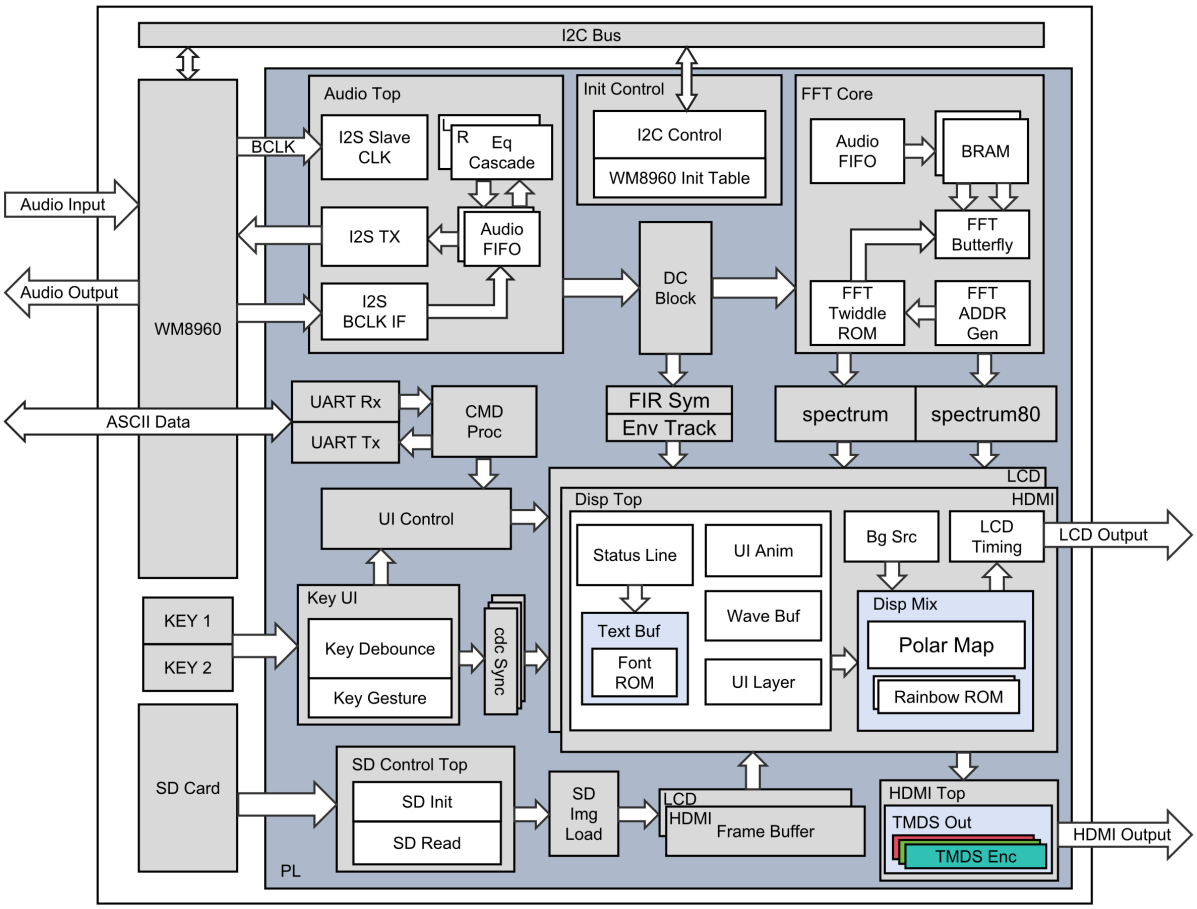


图 2　系统整体架构框图

如上图所示，系统可分为**四大功能区**，以不同底色区分：

表 6　四大功能区的职责与关系

功能区	时钟域	职责
音频处理区	`clk_sys` 48 MHz	I2S 从模式收发、5 段均衡、FFT、频谱分箱；产出“柱高 / 波形”两类可视化数据
显示渲染区	`clk_pix` 12.5 MHz	LCD 扫描与像素合成；吃“坐标 + 柱高 + 波形 + 图片”，吐出 RGB888
HDMI 输出区	`clk_hdmi` 25 MHz	同上渲染核心的第二份实例 + TMDS 编码 + 5:1 串行化
存储与交互区	`clk_sys` 48 MHz	SD 卡协议、图片搬帧、UART 命令、按键手势、配置寄存器组

模块间关系：

- **单向数据流**：音频 → 分析 → 可视化 → 显示。显示层只读分析结果，不反向影响音频处理；
- **统一配置**：所有交互（按键 / 触摸 / UART）都写同一组配置寄存器，再经 `cdc_sync` 分发到各时钟域，保证任一时刻两屏设置一致；
- **跨域汇聚点只有三处**：□ 柱高/波形（`spec_sync` + 格雷码 FIFO）；□ 配置位（`cdc_sync` 双打）；□ SD 读启动（电平翻转 + 双打 + 边沿检测）。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

表 7　硬件组成

部件	型号	说明
主控板	璞致 PA-Starlite	XC7A100T-2FGG484I，200 MHz 差分晶振
音频模块	WM8960	编解码器，I2S 主模式，接 JM2
显示屏	TIANMA TM043NDH02	480×272，40 pin，RGB888，接 JM1
视频输出	HDMI（J5）	**板上无发送芯片**，FPGA 4 对差分经 51 Ω 串阻直连
存储	W25Q128JVSQ	128 Mb QSPI Flash，4 线
存储	microSD	SPI 模式，存图片素材
调试	CH340E	UART → USB，115200 8N1
下载	板载 Digilent JTAG	TypeC

【占位】系统实物全局图（正面）

(此处插入实物正面)

图 3　系统实物全局图（正面）

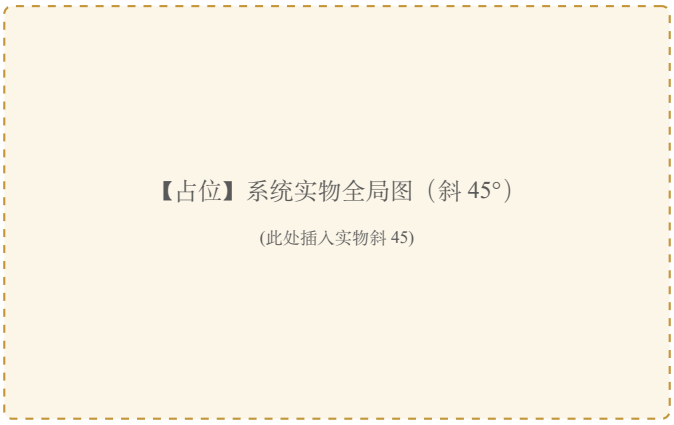


图 4　系统实物全局图（斜 45°）

2.2.2 机械设计介绍

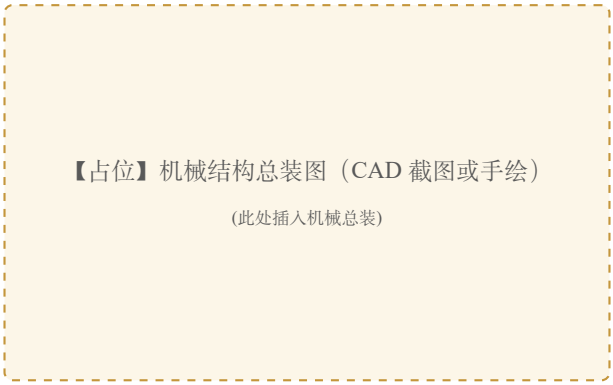


图 5　机械结构总装图（CAD 截图或手绘）

本作品以开发板为核心，无独立机械结构。若后续加装外壳，需要预留的位置为：LCD 排线出口（JM1 侧）、HDMI 接口（J5 侧）、SD 卡插槽、音频模块（JM2）与按键区。

2.2.3 电路各模块介绍

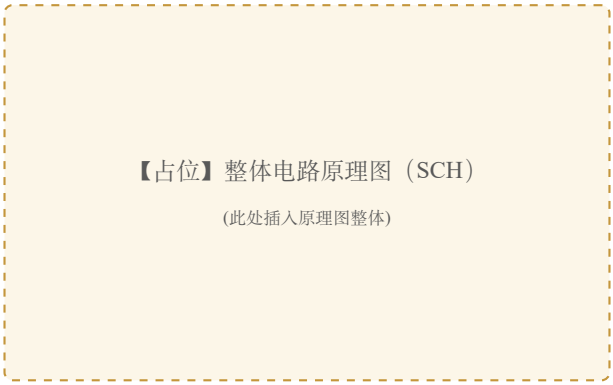


图 6　整体电路原理图（SCH）



图 7　PCB 版图

关键电路模块：

表 8　关键电路模块与信号线

模块	关键输入	关键输出 / 说明
时钟电路	`clk_200m_p/n`（R4/T4，LVDS）	`clk_sys` 48 MHz / `clk_pix` 12.5 MHz / `clk_hdmi` 25 MHz / `clk_5x` 125 MHz
音频接口	`aud_bclk`（3.072 MHz）、`aud_lrclk`（48 kHz）、`aud_adcdat`	`aud_dacdat`；`aud_scl`/`aud_sda`（I2C 配置）
LCD 接口	——	`lcd_rgb[23:0]`、`lcd_hs`、`lcd_vs`、`lcd_clk`（12.5 MHz）
HDMI 接口	`hdmi_hpd`（只进不参与逻辑）	`hdmi_clk_p/n`、`hdmi_d0_p/n`（蓝）、`hdmi_d1_p/n`（绿）、`hdmi_d2_p/n`（红）、`hdmi_out_en`（低有效）
SD 卡接口	`sd_miso`	`sd_clk`、`sd_cs`、`sd_mosi`；LVCMOS33，Bank 14
按键	`key1_n`(W21)、`key2_n`(Y21)、`rst_btn_n`(R14)	均低有效，20 ms 消抖
触摸屏	`tp_dout`	`tp_dclk`、`tp_cs_n`、`tp_din`（XPT2046，SPI）
LED	——	`led[0]` = WM8960 配置完成；`led[1]` = 心跳



图 8　音频模块（WM8960）电路原理图



图 9　HDMI 输出电路原理图（含 51Ω 串阻）

2.3 软件系统介绍

2.3.1 软件整体介绍

本作品的“软件”分两类：**PC 端离线工具**（生成 RTL 需要的表与素材）和**片上手写 RTL**（真正的运行时逻辑）。**全部运行时代码都是 Verilog-2001，无软核、无 HLS、无 C 代码**。

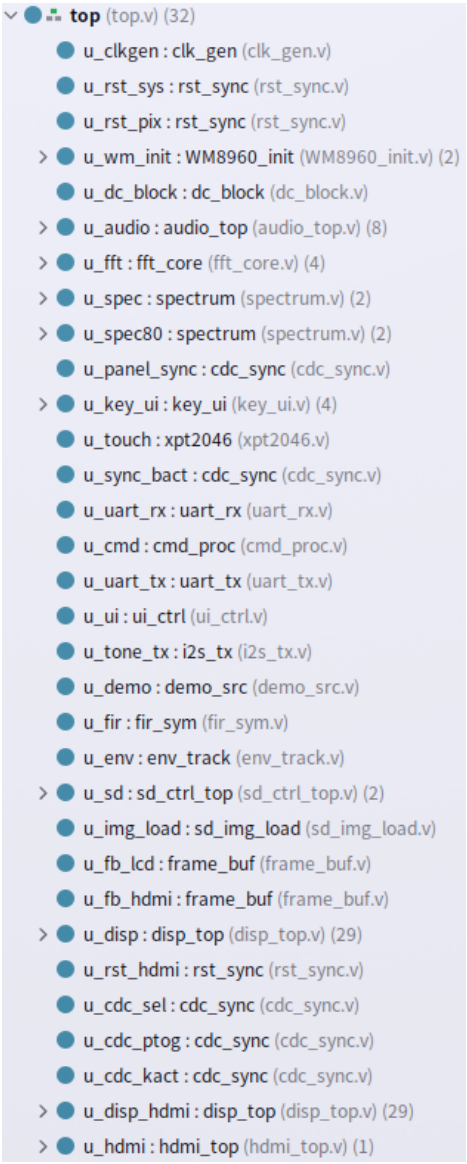

top (top.v) (32)
u_clkgen : clk_gen (clk_gen.v)
u_rst_sys : rst_sync (rst_sync.v)
u_rst_pix : rst_sync (rst_sync.v)
u_wm_init : WM8960_init (WM8960_init.v) (2)
u_dc_block : dc_block (dc_block.v)
u_audio : audio_top (audio_top.v) (8)
u_fft : fft_core (fft_core.v) (4)
u_spec : spectrum (spectrum.v) (2)
u_spec80 : spectrum (spectrum.v) (2)
u_panel_sync : cdc_sync (cdc_sync.v)
u_key_ui : key_ui (key_ui.v) (4)
u_touch : xpt2046 (xpt2046.v)
u_sync_bact : cdc_sync (cdc_sync.v)
u_uart_rx : uart_rx (uart_rx.v)
u_cmd : cmd_proc (cmd_proc.v)
u_uart_tx : uart_tx (uart_tx.v)
u_ui : ui_ctrl (ui_ctrl.v)
u_tone_tx : i2s_tx (i2s_tx.v)
u_demo : demo_src (demo_src.v)
u_fir : fir_sym (fir_sym.v)
u_env : env_track (env_track.v)
u_sd : sd_ctrl_top (sd_ctrl_top.v) (2)
u_img_load : sd_img_load (sd_img_load.v)
u_fb_lcd : frame_buf (frame_buf.v)
u_fb_hdmi : frame_buf (frame_buf.v)
u_disp : disp_top (disp_top.v) (29)
u_rst_hdmi : rst_sync (rst_sync.v)
u_cdc_sel : cdc_sync (cdc_sync.v)
u_cdc_ptog : cdc_sync (cdc_sync.v)
u_cdc_kact : cdc_sync (cdc_sync.v)
u_disp_hdmi : disp_top (disp_top.v) (29)
u_hdmi : hdmi_top (hdmi_top.v) (1)

图 10　工程文件结构

表 9　PC 端离线工具（Python）

脚本	职责 / 产物
golden/dsp.py	滤波器系数定点量化与误差评估 → eq_coeff_rom.v
golden/fft_dsp.py	FFT 旋转因子生成 + 定点误差分析 → fft_twiddle_rom.v
golden/gen_spectrum_map.py	对数频率分箱表（参数化 --nbars）→ spectrum_map.v / spectrum_map80.v
golden/gen_font.py	字模提取 → font_rom.v
golden/gen_wm8960_table.py	WM8960 寄存器表 → WM8960_init_table.v + wm8960_nreg.vh
sdcard/mkimage.py	BMP/PNG/JPG → RGB565 裸扇区（含系统盘保护）
sim/run_iv.sh	39 个 TB 的并行回归（JOBS=nproc）
vivado/build.tcl	综合/实现/比特流（支持 SYNTH_ONLY / IMPL_ONLY）
vivado/program.tcl	SRAM 下载 / QSPI Flash 固化

2.3.2 软硬件划分依据

划分依据是“什么是确定的、什么是易变的”：

表 10　软硬件划分判据

判据	硬件（RTL）	软件（PC 脚本）
时序必须确定	✓	
每次运行结果都一样	✓	
需要人机交互调参	✓	
一次性的数据准备		✓
需要浮点数学		✓
需要第三方库		✓

2.3.3 软件各模块介绍

□ **音频链路（audio_top）**

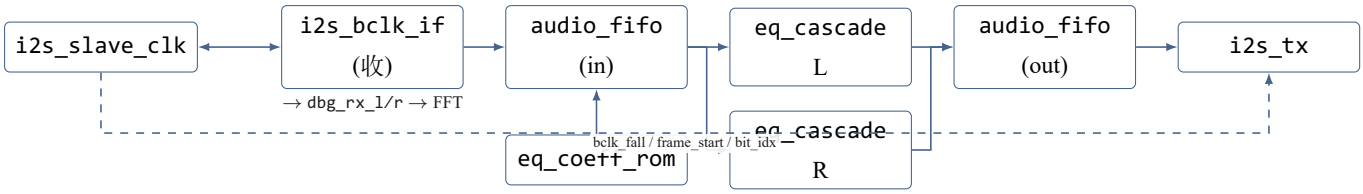


图 11　音频链路数据流（两个声道共享一组系数；FFT 抽头在 EQ 之前）

关键设计：两个 FIFO 分别解耦“BCLK 域 → clk_sys 域”和“EQ 输出速率不固定 → I2S 发送定速”两处速率不匹配。

□ **FFT 引擎（fft_core）**

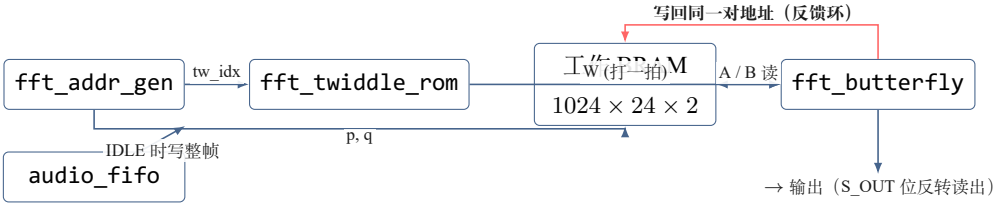


图 12　FFT 数据流：地址发生器同时驱动 BRAM 地址与旋转因子 ROM；蝶形结果写回同一对地址（反馈环）

□ **显示渲染（disp_top，两份实例）**

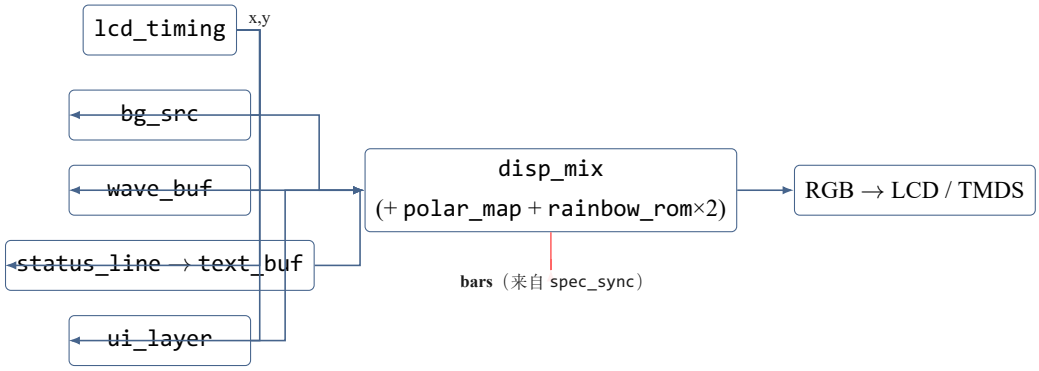


图 13　显示渲染：所有模块吃同一个 (x, y)，各自产出一份颜色/命中信号，最后在 disp_mix 里按优先级叠加

□ **SD 卡图片通道**

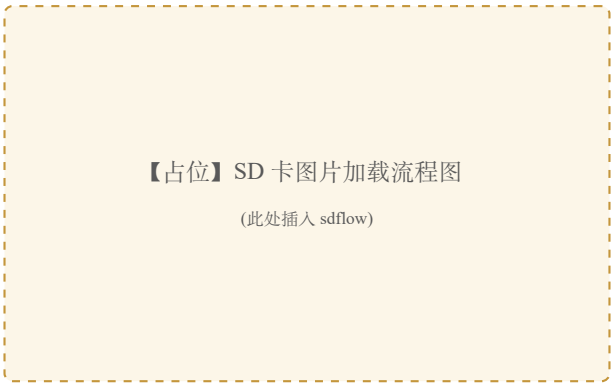


图 14　SD 卡图片加载流程图

流程：`sd_ctrl_top` 切换时钟到 400 kHz → `sd_init` 走完 CMD0/8/55/41 → 切回 25 MHz → `sd_read` 发 CMD18 连续读 510 块 → `sd_img_load` 把字节流拼成 RGB565 并**同时写两份帧缓冲**（全图 480×272 + 中心裁剪 320×180）→ `load_done` 门控切图。

3 第三部分　完成情况及性能参数

3.1 整体介绍



图 15　系统实物照片（正面）



图 16　系统实物照片（斜 45°）

表 11　上板验收清单（全部实测通过）

#	检查项	实测结果
1	仿真回归 `run_iv.sh`	**39/39 PASS**
2	Verible / Verilator	0 违规 / 0 意外警告
3	Vivado 综合 + 实现	**0 CRITICAL / 0 WARNING**
4	`led[1]` 心跳	约 2.9 Hz 闪烁
5	`led[0]`	常亮（WM8960 配置完成）
6	LCD 程序化背景	网格 + 渐变正常显示
7	按 `BG` 键	程序化背景 / SD 图片切换
8	按 `SD` 键	3 张图片循环切换
9	对麦克风说话	频谱柱跳动、波形变化
10	按 `SPK` 键	喇叭静音/取消
11	HDMI 显示器	640×360 有画面（80 柱 + 波形 + 图片）
12	UART 115200	上电打横幅；发 `V` 切视图
13	QSPI Flash 固化	掉电后拔 JTAG 可自启动

3.2 工程成果

3.2.1 机械成果

【占位】机械结构实物照片

(此处插入机械实物)

图 17　机械结构实物照片

说明：本作品以开发板为核心，未做独立外壳。实物包含：主控板、LCD 模组（JM1）、WM8960 音频模块（JM2）、HDMI 线、microSD 卡、TypeC 供电与 JTAG 线。

3.2.2 电路成果



图 18　PCB 实物照片

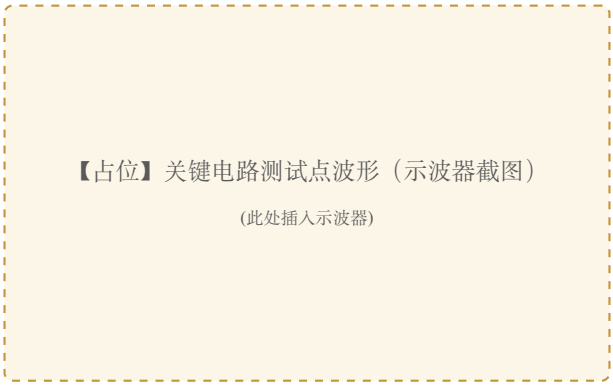


图 19　关键电路测试点波形（示波器截图）

3.2.3 软件成果

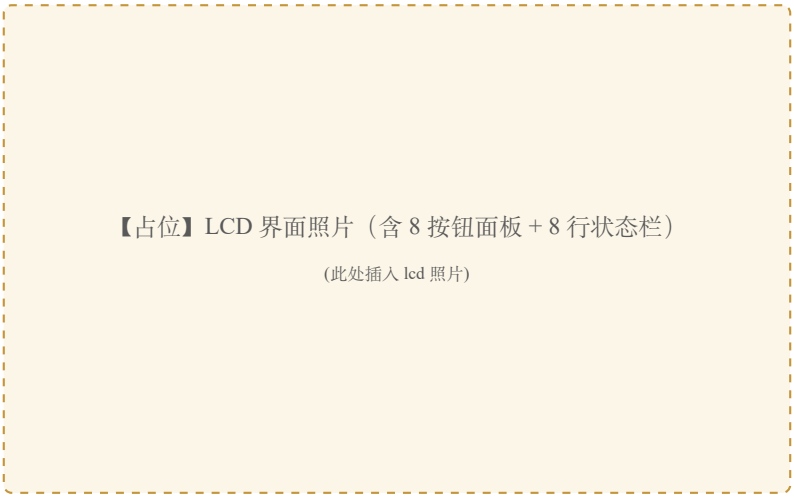


图 20　LCD 界面照片（含 8 按钮面板 + 8 行状态栏）



图 21　HDMI 大屏照片（80 根柱 + 波形 + SD 图片背景）



图 22　UART 命令通道交互截图

3.3 特性成果（量化指标）

□ 优化过程与前后对比

本作品经历 5 个主要优化阶段：

表 12　五个优化阶段的量化收益

阶段	优化内容	关键指标变化
□	同步复位替代异步复位	DPOR-1: 11→**0**；WNS: +1.117→**+2.156 ns**；LUT: 6802→**5648**
□	FFT 全并行 → 单蝶形时分复用	DSP: 2048→**4**（1/512）；BRAM: 0→2
□	新增 HDMI 大屏（80 柱 + 波形）	新增功能，WNS 仍为正
□	SD 图片背景（双帧缓冲方案）	BRAM: 68.5→100.5（**+32 而非 +84**）
□	综合警告清零	警告: 51→**0**

优化 □ 详细对比：

表 13　同步复位优化的前后对比

指标	优化前	优化后	变化
DPOR-1（DSP 异步复位）	11	**0**	✓
WNS	+1.117 ns	**+2.156 ns**	+1.04 ns
Slice LUTs	6 802	**5 648**	−1 154

根因：6 个文件用了 `negedge rst_n`。而 `rst_n` 来自 `rst_sync`（已是域内同步释放），异步复位纯代价——DSP48 内部寄存器只支持同步复位，带异步复位就无法被吸收，额外消耗约 1154 个 LUT 做置位/复位逻辑。□ **教训**：“已修复”必须按根因**全量重扫**（该根因第 4 次复发）。

□ SD 图片背景的四个方案取舍

表 14　SD 图片背景候选方案对比

方案	存储	每份	份数	总 BRAM	WNS	结论
A	640×360×16	103	2	206	——	超器件
B	640×360×8	52	2	134	+0.011	色带过重，实测否定
C	320×180×16	26	1	36.5	+1.84	省资源但 LCD 有黑边
D	**480×272×16 + 320×180×16**	**58+26**	**2**	**100.5**	**+1.78**	**采用**

为什么不用“一个缓冲两个读口”：Vivado 的 BRAM 推断不支持“1 写 2 读跨时钟”，会退化成 LUTRAM 并触发 `DRC UTLZ-1`。

□ 综合警告清零（三类判据）

表 15　51 条警告的分类处理

类型	数量	处理	判据
真缺陷	**40**	**改设计**	`REQP-1839`：`wr2_en`/`wr2_addr` 驱动 BRAM 控制脚却带异步复位
前提不成立	50	豁免（写明判据）	`8-7129`：参数化模块高位地址超出实际使用范围
假报警	1	豁免（附证据）	`Route 35-328`：路由估计未命中，实测 WNS 全为正

□ 功能演示



图 23　柱状频谱视图照片

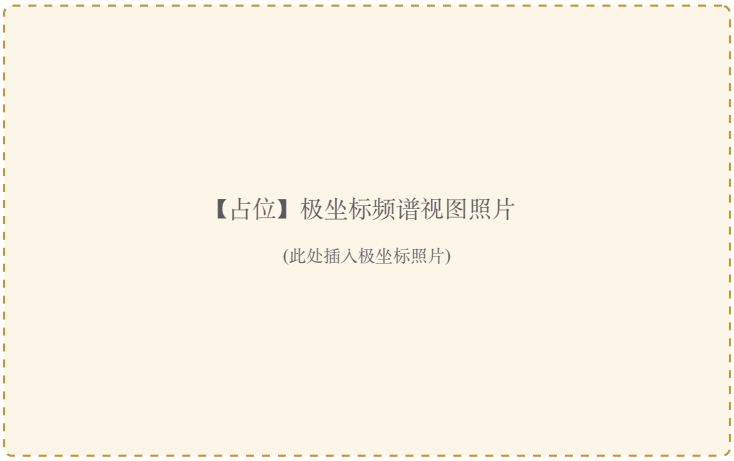


图 24　极坐标频谱视图照片

【占位】动态波形视图照片

(此处插入波形照片)

图 25　动态波形视图照片

【占位】DEMO 演示模式照片

(此处插入 demo 照片)

图 26　DEMO 演示模式照片

4 第四部分　总结

4.1 大模型协作记录

4.1.1 所用 AI 工具与使用环节

表 16　AI 工具与用途

序号	工具	类别	主要用途
1	pi（编码代理 CLI）	智能体框架	工具调用、长会话、文件读写
2	Claude（Anthropic）	主对话模型	设计讨论、RTL 编写、调试、文档
3	exa（MCP）	联网搜索	器件手册、协议细节、第三方实现比对
4	context7（MCP）	库文档检索	Vivado Tcl / Verilator 参数
5	searchcode（MCP）	仓库搜索	找 TMDS 编码器独立实现做交叉验证

表 17　使用环节与具体目的

编号	环节	具体用途
A-01	定点格式设计	Q 格式、位宽预算、舍入与饱和策略
A-02	滤波器结构选择	一阶低架 vs 二阶 peaking、级联顺序、条件数
A-03	FFT 架构权衡	存储式 vs 流式、单蝶形 vs 并行
A-04	跨时钟域方案	格雷码 FIFO vs 双打 vs 电平翻转握手
A-05	显示架构	扫描式渲染 vs framebuffer，参数化边界
B-01	RTL 编写	60 个 Verilog 模块 + 39 个 testbench
B-02	黄金模型	Python 定点模型，逐位对拍
B-03	生成器脚本	旋转因子、频谱分组表、字模、WM8960 寄存器表
B-04	构建流程	`build.tcl` / `program.tcl` / `run_iv.sh`
B-05	调试	读波形、读综合报告、定位时序/资源/DRC 问题
B-06	文档	16 个设计文档 + 825 条问题账本

4.1.2 主要提示方式

方式一：给“判据”而不是“要求”

用户（原始提示词）

实现 SPI 模式的 SD 卡初始化：CMD0 → CMD8 → 循环 (CMD55 + ACMD41) 直到 R1 的 bit0=0。

判据：□ 第一次 Go 必须走完 5 步；□ **第二次 Go 也必须能重新走完**；□ 初始化阶段卡不校验 CRC，用固定值即可。

不要：不要用状态机隐式等待，每步都要有超时计数。

AI 回答（要点）

生成了 5 个状态的状态机（`IDLE/CMD/WAIT/RESP/DONE`），每条命令带 24 位超时计数。首次提交即通过 5/5 判据。**"第二次 Go" 这个判据后来救了大命**——后续调试中发现若 Go 只清 `cnt` 不清 `state`，重配时会永久卡死（`Init_Done` 恒 0 □ 音频被按住复位 □ 切输入源静音 + 频谱冻住）。

方式二：要求"交叉验证"而不是"相信"

用户（原始提示词）

TMDS 编码器我担心记错。请你找 **2~3 份独立实现**（Xilinx 官方示例、开源核、论文附录），**逐行比对**，把差异点列出来，再写。如果三者不一致，停下来告诉我。

结果：找到 3 份实现，其中 2 份在"DC 平衡"分支的 `cnt` 更新条件上写法不同但等价；确认后写入 `tmds_enc.v`，一次通过。

方式三：要求"量化"而不是"看起来对"

用户（原始提示词）

不要给"看起来合理"的结论。要求：□ 实际占用/总量/百分比；□ 与优化前对比；□ 如果变差了，指出是哪一项变差；□ 时序报告里 WNS 是正是负、最差路径在哪。

4.1.3 自我纠错轨迹（五个典型案例）

表 18　自我纠错轨迹

编号	现象	根因	修复 / 沉淀
E-01	改了参数、脚本输出 OK，但上板画面仍居中/颜色错	`str.replace()` 找不到就**静默返回原串**	加 `assert old in s`；账本 807
E-02	合成报 `DRC UTLZ-1`：需 86 452 个 `RAMD64E`	`mem` 被两个时钟读 □ 退化成 LUTRAM	改两份不同尺寸缓冲；账本 789/790/792
E-03	综合报 40 条 `REQP-1839`	`wr2_*` 误放进异步复位块	移出复位块；账本 813
E-04	LCD 与 HDMI 背景不一致	`y[8:3]` 在 360 行时回绕	`YSH = (VDISP>=512)?4:3`；账本 811
E-05	HDMI 图片深蓝显示成暖黄	R/B 互换（LCD 面板是 BGR，反了两次抵消）	HDMI 路径加一次交换；账本 810

4.1.4 智能体工作流设计

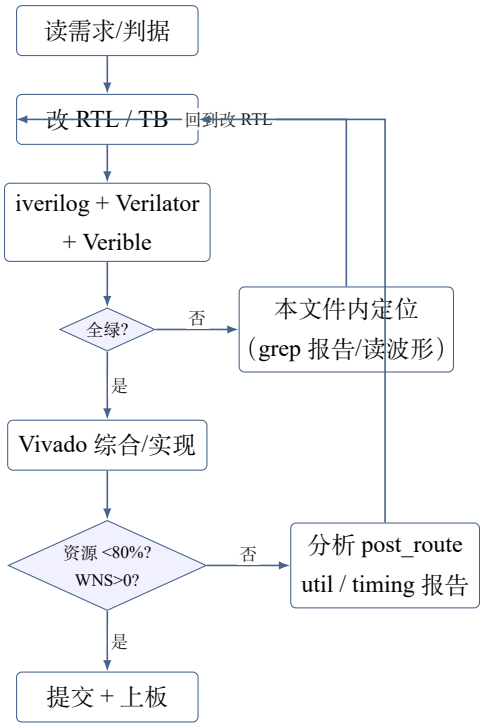


图 27　智能体工作流：每轮“改—验—看报告”，红则定位、绿则推进

关键工作流规则（从失败中总结）：

表 19　工作流规则及其来源

规则	来源
不在红树上继续改	一次改坏后连改 3 次都不中 □ 改成 `git checkout --` 回退重新开始
大改动先建分支	在 `master` 上改到一半 TB 红只能全量回退；建 `fb320` 分支后 WIP 可安全提交
每轮必须跑全量回归	曾因“只跑相关 TB”漏掉 `tp_edges` 的 `UNDRIVEN`
加速先看 user/real 比值	`run_iv.sh` 20m54s→1m30s：瓶颈不是并行度，是单个 `tb_top` 吃 20 分钟
GUI 能做的事 = 一串 Tcl	Flash 烧写 CLI 失败 7 轮 □ 照抄 GUI 的 Tcl Console □ 一次成功
连猜 3 次不中 □ 停止猜测	HDMI 撕裂猜了 4 次（复位延迟/2.6 ms/locked/OSERDESE2）□ 改文档记录 + 待 ILA

4.1.5 采纳、人工修改与核验

表 20　AI 输出的采纳情况与核验

编号	采纳	人工修改	核验结果
A-01	采纳	无	✓逐位一致
A-02	部分	结构从 4 段改 5 段，加 8 kHz peaking	✓频响误差 <0.5 dB
A-03	采纳	无	✓2048 频点逐位一致
A-04	部分	格雷码 FIFO 改“电平翻转 + 双打 + 边沿检测”	✓4/4 PASS
A-05	部分	加 `WITH_UI`/`WITH_TEXT`/`FB_*` 参数族	✓130 560 像素逐点一致
B-01	采纳	多次人工纠错（见 E-01~E-05）	✓39/39
B-02	采纳	无	✓逐位一致
B-03	采纳	`gen_spectrum_map.py` 后参数化	✓60/80 均验证
B-04	部分	Flash 烧写改“先 `program_hw_devices` 建 SPI 通道”	✓43 秒成功
B-05	采纳	无	✓WNS 全为正
B-06	采纳	全部数字人工核对	✓与报告一致

核验方法论：□ 逐位对拍（不允许“看起来差不多”）；□ 独立真值（`numpy.fft` / `$atan2` / `$sqrt`）；□ 穷举（512 个 bin 分组、5132 组地址、49 920 个像素）；□ 总量守恒（一帧 150 150 拍、130 560 个有效像素）；□ 读回验证（帧缓冲逐点写入再读回）。

4.2 技能包的提炼过程

4.2.1 三级沉淀体系

表 21　三级知识沉淀

层级	载体	规模	作用
问题账本	`docs/08-skills-index.md`	**825 条**	现象/根因/修复/量化改善，原始可检索
技能包	`.pi/skills/*/SKILL.md`	**4 个**	触发条件/方法/判据/边界，可复用
状态快照	`docs/14-status-snapshot.md`	1 份	一眼看清“能用的/受限的/待办的”

4.2.2 四个技能包的来源、验证与边界

表 22　技能包一览

技能包	来源失败	如何验证有效	边界
`fixed-point-dsp`	EQ 频响差 3 dB（位宽不足）；`WAVE_AMP` 改大反而变小（静默截断）；某频点发散（条件数）	重做 EQ，频响误差 3 dB→**0.5 dB**；截断问题后续 6 个参数**零复发**	适用：定点 IIR/FIR/FFT，16~32 bit。不适用：浮点、位宽 <12 bit
`fpga-rtl-flow`	找不到 Vivado；DRC 在 GUI 不报；时序违例方向错；`place` 失败伪装成综合错；构建 9m38s 太慢	构建 **9m38s→2m57s**；回归 **20m54s→1m30s**；后续 20+ 次构建**零次返工**	适用：纯 RTL、无 block design。不适用：IP integrator、部分重配置
`rtl-verification`	在信号自己更新的沿上读它；非阻塞赋值读不到新值；跨帧/帧内状态共用复位；只测一次漏掉重配路径	39 个 TB 中**至少 12 个第一次跑就红**；账本里**大部分失败是仿真发现不了的**，但那 12 条都是上板必花几天的	适用：确定性数字逻辑。不适用：模拟/混合信号
`contributor`	重复劳动、项目不活跃、未读 CONTRIBUTING	——	适用：活跃开源项目

4.2.3 从“观察”到“技能”的抽象链路

以 rtl-verification 为例

原始观察（账本 412）：TB 里 `@(negedge sd_clk)` 采样，总差一拍。

↓追问“为什么”

根因：`sd_clk` 由 `div_clk` 分频而来，`negedge sd_clk` 恰好等于 `posedge div_clk`，读到的是旧值。

↓再问“是个例还是通类”

聚类（账本 413/414/415）：非阻塞赋值在同一 `always` 读不到新值；`acmd41_cnt` 在 CS 沿被清零导致永远回不到第 3 次；边际推理调不动时应换显式计数。

↓抽象成规则

技能包条目：“永远不要在信号自己更新的沿上读它”；“复位的作用域要精确界定：跨帧状态 ≠ 帧内状态”。

↓写成可执行检查

判据：写 TB 时自问——这个 `@(posedge/negedge X)` 和 X 的驱动沿是同一个吗？是则改成 `@(posedge clk); #1;` 或换一个沿。

4.2.4 边界如何确定

判据：每条技能都必须能回答三个问题——□ 它解决什么现象？□ 它的前提是什么？□ 前提不成立时会怎样？

反面例子（避免把特例当通例）：

× **不要写**：“BRAM 一定要加 `(* ram_style = "block" *)`”——实测该属性在“1 写 2 读跨时钟”下**无效**。

✓**应该写**：“BRAM 推断由**访问模式**决定，属性只是建议。1W1R 可靠；1W2R 跨时钟必退化成 LUTRAM，属性也救不了。”

4.3 可扩展之处

表 23　可扩展方向

方向	现状	扩展思路
HDMI 放大平滑	320×180 直接 2× 放大，可见方块边界	加水平双线性：奇数列取 $(p_{sx}+p_{sx+1})/2$，只需 1 个寄存器（方案见 `docs/16`）
线路输入	代码就绪，板上 C4(1 μF) 开路	补焊后即可双声道线路输入
HDMI 撕裂	复位后偶发右半屏撕裂	需 ILA 抓 TMDS 或示波器测 5× 时钟相位
外部存储	目前图片存 BRAM（74%）	接 DDR3/SDRAM 可支持 1080p 满帧缓冲；需手写控制器以保持“零 IP”
触摸交互	XPT2046 已驱动，但 LCD 模组无触摸层	换 `TM043NBH02` 即可支持点按操作

4.4 心得体会

一、写 RTL 最难的不是“写”，是“验证”

项目初期我以为难点在 RTL 本身。做完 60 个模块后才发现，**真正耗时的是“怎么知道它是对的”**。最后的做法是：每个模块配一个 Python 定点黄金模型，逐位对拍，不允许“波形看起来差不多”。39 个 testbench 里 **至少 12 个在第一次跑就红**——而这 12 条每条都对应上板时可能花掉一两天的 bug。

印象最深的是 `sd_img_load`：SD 一块是 262 144 字节，而一帧只需 261 120 字节，多出的 1024 字节反复写最后一个地址，把画面右下角踩坏。这个 bug **仿真能抓到、上板极难定位**（表现为“右下角一小块花屏”）。这就是“逐位对拍”的价值。

二、资源永远比想象中紧

640×360×16 bit 的帧缓冲算下来要 103 块 BRAM，而器件只有 135 块。我以为“留 30 块给其它模块够了”，实测发现加上其它器件直接到 98%，**布线拥塞导致时序崩到 −1.09 ns（`place_design` 直接失败）**。

后来走了四步：□ 640×360×8（8 位色）——能构建，但 256 色在渐变图上看得到明显色带，被用户否掉；□ 320×180 单份（LCD 居中）——省资源，但 LCD 有黑边；□ 最后用**两份不同尺寸的缓冲**：LCD 480×272 全图 + HDMI 320×180 裁剪区，**总 BRAM 100.5（74%）**，时序 +1.78 ns。

最大收获：存储压力 = 像素数 × 位深，是两个旋钮。我一开始只拧了“位深”这一个，走了一大圈才发现“降分辨率 + 整数倍放大”更好。

三、“能构建” ≠ “能交付”

时序违例**不会**阻止 Vivado 生成 `.bit`——它只会报一个负的 WNS。所以判断标准不能是“构建成功”，必须是“**所有时钟组的 WNS 都为正**”。有一次我因为只看返回码 0 就以为成功，差点把一个 $\mathrm{WNS} = -0.82\,\mathrm{ns}$ 的比特流烧上板。

四、调试的“三级跳”

1. **第一级：看报错**——工具说得清清楚楚的问题（语法、端口不匹配）不要浪费时间猜；
2. **第二级：看报告**——综合/实现的 util、timing、DRC、CDC 报告里，工具其实已经把根因写出来了（如“LUT as Distributed RAM over-utilized”直接说明存储器没映射到 BRAM）；
3. **第三级：改设计**——修根因，不要修症状。同一根因可能在不同模块复发（同步复位问题复发了 4 次），所以**修完要全量重扫**。

同一问题连猜 3 次不中 □ 停止猜测，换成能观测内部状态的手段。HDMI 撕裂我猜了 4 次（复位延迟 256 周期 / 2.6 ms / locked AND / OSERDESE2），全部无效。最后还是把它写进文档、标“已试过 × 无效”，留给 ILA 或示波器——**知道什么时候该停，也是能力**。

五、关于用大模型写 RTL

我的体会是：**大模型擅长“把已知方案写出来”，不擅长“判断方案对不对”**。

- 让它写一个 TMDS 编码器——写得又快又好，因为这是有标准答案的；
- 让它判断“该用一份缓冲双读口还是两份缓冲”——它会给出一个“看起来合理”的答案，而真正正确的答案要靠**我自己去算 BRAM 数量、去读 UG473、去跑综合验证**；
- 最容易出问题的是**批量修改**：`str.replace()` 找不到就静默成功，改完脚本报“OK”但文件根本没变。所以现在所有批量替换都强制加 `assert`。

正确的用法是把它当“高速打字机 + 交叉验证工具”，而不是“决策者”。所有关键决策（架构、位宽、时序预算）必须自己算过、自己验证过。

六、写在最后

825 条问题账本看起来很多，但每一条背后都是一次真实的失败。这个项目最大的资产不是那 11 716 个 LUT，而是这份账本——它让我在第二个类似项目里，**避开了 80% 曾经踩过的坑**。

5 第五部分　参考文献

1. Xilinx. *7 Series FPGAs Memory Resources (UG473)*. 2019.
2. Xilinx. *7 Series FPGAs SelectIO Resources (UG471)*. 2018.
3. Xilinx. *7 Series FPGAs Clocking Resources (UG472)*. 2018.
4. Xilinx. *Vivado Design Suite User Guide: Using Constraints (UG903)*. 2020.
5. Xilinx. *UltraScale Architecture Libraries Guide (UG974)* —OSERDESE2/OBUFDS.
6. Digital Display Working Group. *Digital Visual Interface (DVI) Specification, Rev 1.0*. 1999.
7. HDMI Licensing, LLC. *HDMI Specification Version 1.4b*. 2011.
8. Wolfson Microelectronics. *WM8960 Data Sheet (Rev 4.2)*. 2013.
9. SD Association. *SD Specifications Part 1: Physical Layer Simplified Specification v6.00*. 2017.
10. Winbond. *W25Q128JV 3V 128M-bit Serial Flash Memory Datasheet*. 2018.
11. J. W. Cooley, J. W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 1965, 19(90): 297–301.
12. A. V. Oppenheim, R. W. Schafer. *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
13. S. W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing, 1997.
14. R. G. Lyons. *Understanding Digital Signal Processing*, 3rd ed. Prentice Hall, 2010.
15. C. E. Shannon. Communication in the Presence of Noise. *Proceedings of the IRE*, 1949, 37(1): 10–21.
16. M. Keating, et al. *Low Power Methodology Manual*. Springer, 2007.
17. C. E. Cummings. Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog. *SNUG*, 2008.
18. C. E. Cummings. Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs. *SNUG*, 2001.
19. 田耘, 徐文波. *Xilinx FPGA 开发实用教程*. 清华大学出版社, 2008.
20. 吴厚航. *深入浅出玩转 FPGA*. 北京航空航天大学出版社, 2010.

A 附录：复现说明

1.1 环境要求

表 24　软件环境

工具	版本	说明
Vivado	2020.2	免费 WebPACK 版即可（7A100T 在支持列表内）
iverilog	12.0	`pacman -S iverilog` / `apt install iverilog`
Verilator	5.052	`pacman -S verilator`
Verible	latest	`mason` 或 GitHub release
Python	≥3.10	系统自带
Pillow	≥10.0	仅 `mkimage.py` 处理 PNG/JPG 时需要
中文字体	Noto CJK	编译本文档需要

表 25　硬件环境

项	规格
开发板	璞致 PA-Starlite（XC7A100T-2FGG484I）
音频模块	WM8960（接 JM2）
显示屏	TIANMA TM043NDH02（480×272，40 pin，接 JM1）
HDMI 显示器	任意支持 640×360@60 的显示器
microSD 卡	≥1 GB，需要写卡权限（`sudo`）
下载器	板载 Digilent JTAG（TypeC）

1.2 从零开始的完整步骤

1.2.1 步骤 1：获取代码

```
git clone https://github.com/flyingyuya/Sonic-Speed-Parallelism.git
cd Sonic-Speed-Parallelism
```

1.2.2 步骤 2：工具链自检

```
bash scripts/sim/run_iv.sh tools
# 预期：[ok] iverilog / verilator / verible 三行
# 若 Vivado 找不到：export PATH=$HOME/Xilinx/Vivado/2020.2/bin:$PATH
```

1.2.3 步骤 3：全量仿真（约 1 分 30 秒）

```
bash scripts/sim/run_iv.sh
# 预期：通过 39  失败 0  跳过 0    <-- 环境是否正确的黄金标准
# 单项：bash scripts/sim/run_iv.sh fft_core
# 含 tb_top 的全量：FULL=1 bash scripts/sim/run_iv.sh
```

1.2.4 步骤 4：静态检查

```
verible-verilog-lint --rules_config_search $(find rtl sim -name '*.v')
# 预期：无输出（退出码 0）
```

1.2.5 步骤 5：综合与实现（约 9~10 分钟）

```
vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
# 预期：构建 0  CRITICAL 0  WARNING 0
#       LUT 11716 (18.48%)  BRAM 100.5 (74.44%)
#       clk_sys +1.779ns    clk_hdmi +4.028ns
# 只综合（约 3 分钟，看资源够不够）：
SYNTH_ONLY=1 vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
```

1.2.6 步骤 6：准备 SD 卡（需要 sudo）

```
lsblk -o NAME,SIZE,TYPE,TRAN | grep -E "sd|mmc"     # 找到读卡器，如 sda

# 先 dry-run（强烈建议）
python3 scripts/sdcard/mkimage.py --dry-run /dev/sda img1.png img2.png img3.png
# 预期每张图 261120 字节 = 480x272x2  <-- 数字对了就说明格式正确

# 正式写入（脚本内建系统盘保护）
echo YES | sudo python3 scripts/sdcard/mkimage.py /dev/sda img1.png img2.png img3.png
```

1.2.7 步骤 7：上板

```
# 1) 验证 JTAG
vivado -mode batch -source scripts/vivado/program.tcl -tclargs detect
# 预期：发现 JTAG 目标 ... 认到目标器件 xc7a100t

# 2) 下载到 SRAM（10 秒，掉电即失，开发期用这个）
vivado -mode batch -source scripts/vivado/program.tcl

# 3) 固化到 Flash（43 秒，掉电保持，定型后才用）
vivado -mode batch -source scripts/vivado/program.tcl -tclargs flash
# 预期：Mfg ID ef / Erase-Program-Verify successful
# 注意：JTAG 优先于 QSPI，验证 Flash 启动必须拔掉 JTAG 线再上电
```

1.3 常见问题

表 26　常见问题与解决

现象	原因	解决
`run_iv.sh` 全失败	iverilog 太老或文件清单引用了已删文件	需 ≥11.0；`grep -rn <文件名>` 扫一遍清单
综合报 `UTLZ-1`（LUTRAM 超限）	存储器多端口访问模式不匹配推断模板	复制实例，不要加读口
`place_design` 失败但无时序错	BRAM >95% 导致布线拥塞	看 `post_route_util.rpt`，降位深/降分辨率
上板 LCD 全白/全黑	LCD 时序参数错	跑 `run_iv.sh lcd_timing`
SD 图片乱码/斜切	卡里数据尺寸与 RTL 的 W/H 不匹配	重新写卡，确认每图 261 120 字节
HDMI 颜色偏黄	R/B 通道互换	LCD 面板是 BGR，HDMI 路径需再换一次
HDMI 复位后偶发右半屏撕裂	已知未解决（TMDS 层）	重按复位可恢复；需 ILA/示波器
线路输入无声	板上 `C4`(1 μF) 开路	需补焊（代码已就绪）